

Same Last-Item Confusion Unveiled: A Unified Mitigation Framework for Graph Learning in Session-Based Recommendation

Jinpeng Chen^{*†}

School of Computer Science (National Pilot Software Engineering School),
Beijing University of Posts and Telecommunications
Beijing, China
jpchen@bupt.edu.cn

Jianxiang He[†]

School of Computer Science (National Pilot Software Engineering School),
Beijing University of Posts and Telecommunications
Beijing, China
hjsx812143280@bupt.edu.cn

Yuan Cao

National University of Singapore
Singapore, Singapore
e1124923@u.nus.edu

Huan Li

The State Key Laboratory of Blockchain and Data Security,
Zhejiang University
Hangzhou, China
lihuan.cs@zju.edu.cn

Zhenye Yang

Beijing University of Posts and Telecommunications
Beijing, China
yangzhenye@bupt.edu.cn

Kaimin Wei

College of Information Science and Technology, Jinan University
Guangzhou, China
kaiminwei@jnu.edu.cn

Xiongnan Jin

Shenzhen University
Shenzhen, China
xiongnanjin@szu.edu.cn

Senzhang Wang

Central South University
Changsha, China
szwang@csu.edu.cn

Weiping Tu

TravelSky Technology Limited
Beijing, China
wptu@travelsky.com.cn

Abstract

Session-based recommendation (SBR), which focuses on next-item prediction for anonymous users based on short-term interaction sequences, has garnered increasing attention from researchers. While graph neural networks (GNNs) have become predominant in modeling complex item transition patterns, our empirical study reveals two critical limitations in existing GNN-based SBR methods. On the one hand, they struggle to differentiate between sessions sharing the same last item, resulting in indistinguishable session representations. On the other hand, the inherent popularity bias in session data leads to the over-recommendation of popular items. Inspired by contrastive learning techniques, this paper presents a unified mitigation framework for Same Last-item confusion in Graph Learning (SAGE) for SBR. In SAGE, we first obtain normalized session embeddings on constructed session graphs. We then build positive and negative samples of sessions through dual forward propagations and a novel negative sample selection strategy, followed by calculating contrastive loss. Finally, the enhanced session embeddings are utilized for prediction. Extensive experiments on two real-world

datasets demonstrate that integrating SAGE with various state-of-the-art GNN-based SBR methods significantly improves their original performances.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Recommender systems, contrastive learning, session-based recommendation

ACM Reference Format:

Jinpeng Chen, Jianxiang He, Yuan Cao, Huan Li, Zhenye Yang, Kaimin Wei, Xiongnan Jin, Senzhang Wang, and Weiping Tu. 2026. Same Last-Item Confusion Unveiled: A Unified Mitigation Framework for Graph Learning in Session-Based Recommendation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792356>

1 Introduction

With the progressive development of the Internet and the explosion of online information, recommender systems have become essential to the digital ecosystem. Sequential recommender systems [3, 8, 9, 22], in particular, capture user preferences' evolving nature by considering user-level and item-level information. However, in some scenarios, users may remain anonymous [16], meaning that only item-level features are available, and user-level information is not accessible. In such cases, session-based recommenders (SBR), focusing solely on the current session rather than

^{*}Corresponding author.

[†]Also with Key Laboratory of Trustworthy Distributed Computing and Service (BUPT), Ministry of Education.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792356>

relying on extensive historical user-item interactions, have gained increasing attention in recent years [4, 5, 31, 42].

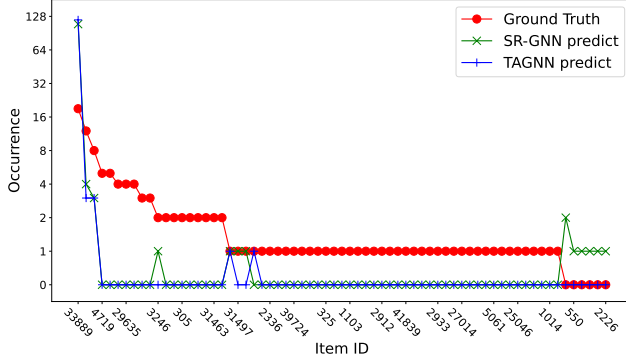


Figure 1: The “Same Last-Item Confusion” Example. We trained SR-GNN and TAGNN SBR models on the Diginetica dataset and selected a set of sessions (127 in total) with the same last-interacted item (*item 33889*, the most commonly occurring last item) for prediction. The line graph tracks the occurrences of different items in the ground truth and prediction. The results reveal a tendency for these GNN methods to predict the same item (*item 33889*) for sessions that share the same last item, despite the ground truth showing the underlying prediction diversity in those sessions.

To capture the sequential relationship in user-item interactions, Markov Chain (MC)-based sequential recommenders [13, 27] were proposed. However, due to their limited representational capacity and strong reliance on the last interacted item in each session, their performance remains constrained. Recurrent neural networks (RNNs) were then introduced into session-based recommendation thanks to their natural ability to model sequential information. With the introduction of GRU4Rec [18], RNNs turned out to be the structure of choice for solving the session recommendation problem. For example, NARM [19] designs two RNNs to capture users’ global and local sequential preferences correspondingly. However, it was not until the existence of SR-GNN [37], which utilized a gated graph neural network (GGNN) to extract sequential information on a session graph, that significant improvements were observed. Since then, works following the basic schema of SR-GNN have been proposed. For example, TAGNN [41] added a target-attentive mechanism to SR-GNN, achieving promising results. Besides, as SR-GNN focuses on a local session, methods like FGNN [25] and GCE-GNN [35] incorporate global session information in different ways, further enhancing SBR. Disen-GNN [17] disentangles item factors and uses factor-level attention to better capture user intent. DMI-GNN [23] applies multi-positional encoding with dynamic regularization.

Although all methods mentioned above have achieved favorable results, they still inherit a major limitation from the MC-based approach: a strong reliance on the last interacted item. Take SR-GNN as an example; when assembling the final session representation, it simply applies a linear transformation over the concatenation of the embedding vector of the last item and the global embedding vector.

As a result, these methods may struggle to distinguish between sessions that share the same last interacted item. To illustrate this, we trained SR-GNN and TAGNN on the Diginetica dataset and selected 127 sessions that concluded with the same last-interacted item, *item 33889*—the most frequently occurring last item in the dataset. As shown in Fig. 1, the line graph compares the number of occurrences of various items in the ground truth against the predictions generated by the GNN-based models. The results reveal a tendency for methods employing this technique to predict the same item for sessions that share the same last item, which limits the underlying prediction diversity in those sessions. As mentioned earlier, a large branch of existing graph-based methods ([35, 41], for example) refers to the same approach as SR-GNN in the final assembly of the session embedding, resulting in this phenomenon being widespread in current advanced graph-based SBR methods. In this paper, we refer to this phenomenon as the “same last-item confusion” problem. Apart from this, Fig. 1 also supports the hypothesis proposed by [12] that SR-GNN predictions are susceptible to popularity bias.

To address the “same last-item confusion” problem and mitigate popularity bias across various GNN-based SBR methods, we propose a unified mitigation framework for same last-item confusion in graph learning for SBR. Inspired by the success of contrastive learning in recommendation systems [11, 33, 36], we aim to mitigate the confusion caused by the last interacted item by increasing the discrepancy between sessions with the same last item, treating those sessions as negative samples. However, defining positive samples is challenging, so we follow the approach proposed by [10] by treating the session itself as a positive sample through dropout [29] techniques. To this end, we design a contrastive module that increases the discrepancy between sessions with the same last interacted item. Furthermore, to counteract the influence of popular items in the final prediction and better capture the user’s underlying interests, we normalize both item and session embeddings.

Our main contributions in this paper are listed as follows.

- We propose *SAGE*, a unified framework to address the “same last-item confusion” problem in GNN-based SBR methods using contrastive learning with a novel hard negative sampling strategy.
- We mitigate popularity bias by introducing normalized item embeddings and session embeddings.
- We validate *SAGE* through extensive experiments, showing significant performance improvements and effectiveness in resolving both “same last-item confusion” and popularity bias.

2 Related Work

2.1 Traditional SBR Methods

A series of approaches [15, 24, 28] based on matrix factorization (MF) are representatives of the traditional methods. MF-based methods consider that the user-item interaction matrix is too sparse and then decompose the matrix into two low-rank dense matrices, which correspond to users and items, respectively. However, MF-based approaches do not capture users’ sequential interaction behavior, making them unsuitable for sequential recommendation tasks.

Given that static matrix decomposition methods do not model sequence behavior, FPMC [27] models user behavior as a Markov Chain and combines Markov Chains with MF. FOSSIL [13] improves on FPMC by introducing factorized sequential prediction with an item similarity model and higher-order Markov Chains, thus both long-term and short-term user behavior are taken into account. However, due to the expressive capability limitation of their shallow networks, they do not perform well in nowadays more complex recommendation scenarios.

2.2 Deep Learning-Based SBR Methods

In recent years, as deep-learning methods emerge, deep learning-based recommendation methods [6, 30] utilizing recurrent neural networks (RNNs) are on the rise. GRU4Rec [18] is a typical pioneer in utilizing RNN structures for making sequential recommendations. Subsequently, more attempts have been made to perform sequential recommendations using RNNs. Tan et al. [32] improved the performance of RNN recommendation models by proposing several data enhancements and training tricks. Li et al. [19] proposed a neural attentive recommendation machine with an encoder-decoder structure based on RNNs to capture the user's sequential preferences. STAMP [21], which replaces the RNN structure with simple MLPs and attention mechanisms, proved to be efficient in capturing both users' static and dynamic interests.

Although deep learning-based models have more powerful representation capabilities than traditional methods, these approaches still cannot effectively model complex item relationships, such as non-adjacent item transitions.

2.3 Neural Networks on Graphs

Recently, with the rise of graph neural networks [1, 7, 34], many approaches [2, 14, 40] have utilized GNN-based network structures to address SBR problems. SR-GNN [37] is one of the earliest and most representative models, which models each session as a session graph and applies a gated-GNN [20] to obtain a representative embedding of the session. Following the success of SR-GNN, many variants have been proposed. For example, TAGNN [41] introduced a target-aware attention mechanism to SR-GNN, which adaptively activates different user interests concerning varied target items. FGNN [25] takes both sequence order and latent order in the session graph into consideration. S²-DHCN [39] leverages hypergraph techniques to represent each session and utilizes contrastive learning techniques to perform self-supervised learning. Additionally, methods like GCE-GNN [35] extend beyond local session graphs by constructing global graphs to capture broader dataset-wide information. Disen-GNN [17] enhances SBR by disentangling item embeddings into independent factors and leveraging factor-level GNNs and attention to capture user intents. DMI-GNN[23] introduces multi-positional encoding and dynamic regularization to mitigate excessive interests in SBR.

However, as discussed in the introduction, all of these methods suffer from the "same last-item confusion" problem and are affected by popularity bias, leading to performance degradation. Despite various enhancements, the fundamental issues of differentiating sessions with the same last item and mitigating the dominance of popular items remain largely unaddressed.

3 Methodology

3.1 Problem Formulation

A session-based recommender aims to offer recommendations to users based on the inputs of their anonymous historical interaction sequences, e.g., clicks or purchases. Given the item set $\mathcal{V} = \{v_1, v_2, \dots, v_m\}$, an anonymous interaction session s is an item sequence $s_i = [v_1^i, v_2^i, \dots, v_{|s_i|}^i]$, $s_i \in \mathcal{S}$, where v_j^i is the j -th interacted item of the i -th session. Given session s_i , the goal is to predict $v_{|s_i|+1}^i$. The important notations used in this paper are detailed in Appendix A.

3.2 Overview

The overall workflow of our proposed *SAGE* is illustrated in Fig. 2. *SAGE* consists of four main modules: Session Feature Extraction Module, Prediction Module, Same Last-Item Confusion Mitigation Module, and Optimization Module. Initially, the original sessions are transformed into session graphs, from which graph-based item representations are extracted using GNN. Attention networks and normalizations are then employed to generate the final hybrid session representation. The Session Feature Extraction Module undergoes two forward propagations. During propagation, passing through the Dropout layer yields two distinct session representation vectors for the same session. The first representation facilitates a machine factorization (MF)-like item prediction. Meanwhile, negative samples are sampled from a memory bank, and the two generated session representations serve as the query and key for calculating the contrastive loss.

3.3 Session Feature Extraction Module

3.3.1 Normalized Item Embedding. Since the one-hot vectors of items are sparse and high-dimensional and do not carry pairwise distance information, we first embed each item $v_i \in \mathcal{V}$ into a d -dimensional representation $\mathbf{v}_i \in \mathbb{R}^d$.

As some existing methods directly utilize $\mathbf{v}_i$ for downstream tasks, we argue that the direct use of embedded vectors would lead to popularity bias [12]. To this end, we intuitively add L_2 normalization to the raw feature vector. Apart from this, we also add Dropout directly on an embedding layer for the downstream contrastive module. The normalized vectors $\hat{\mathbf{v}}_i$ are given as follows:

$$\hat{\mathbf{v}}_i = \text{Dropout}(\text{Norm}(\mathbf{v}_i), p), \quad (1)$$

where p is the dropout probability, $\text{Norm}(\cdot)$ is the L2-normalization function.

3.3.2 Session Graph Construction. To explore rich transitions among items and generate accurate latent vectors of items, as shown in Fig.3, we model each session s as a directed graph $\mathcal{G}_s = (\mathcal{V}_s, \mathcal{E}_s)$, where each node $v_{s,i} \in \mathcal{V}_s$ represents an item and each edge $(v_{s,i-1}, v_{s,i}) \in \mathcal{E}_s$ represents a user interaction where $v_{s,i}$ is interacted with after $v_{s,i-1}$. This session graph construction approach is consistent with the method used in SR-GNN and has been widely adopted by the majority of GNN-based session-based recommendation methods. To handle items that occur multiple times within sessions, we assign a normalized weight to each edge. The weight is calculated by dividing the occurrence of the corresponding interaction $((v_{s,i-1}, v_{s,i}))$ by the out-degree of the starting node $(v_{s,i-1})$. We

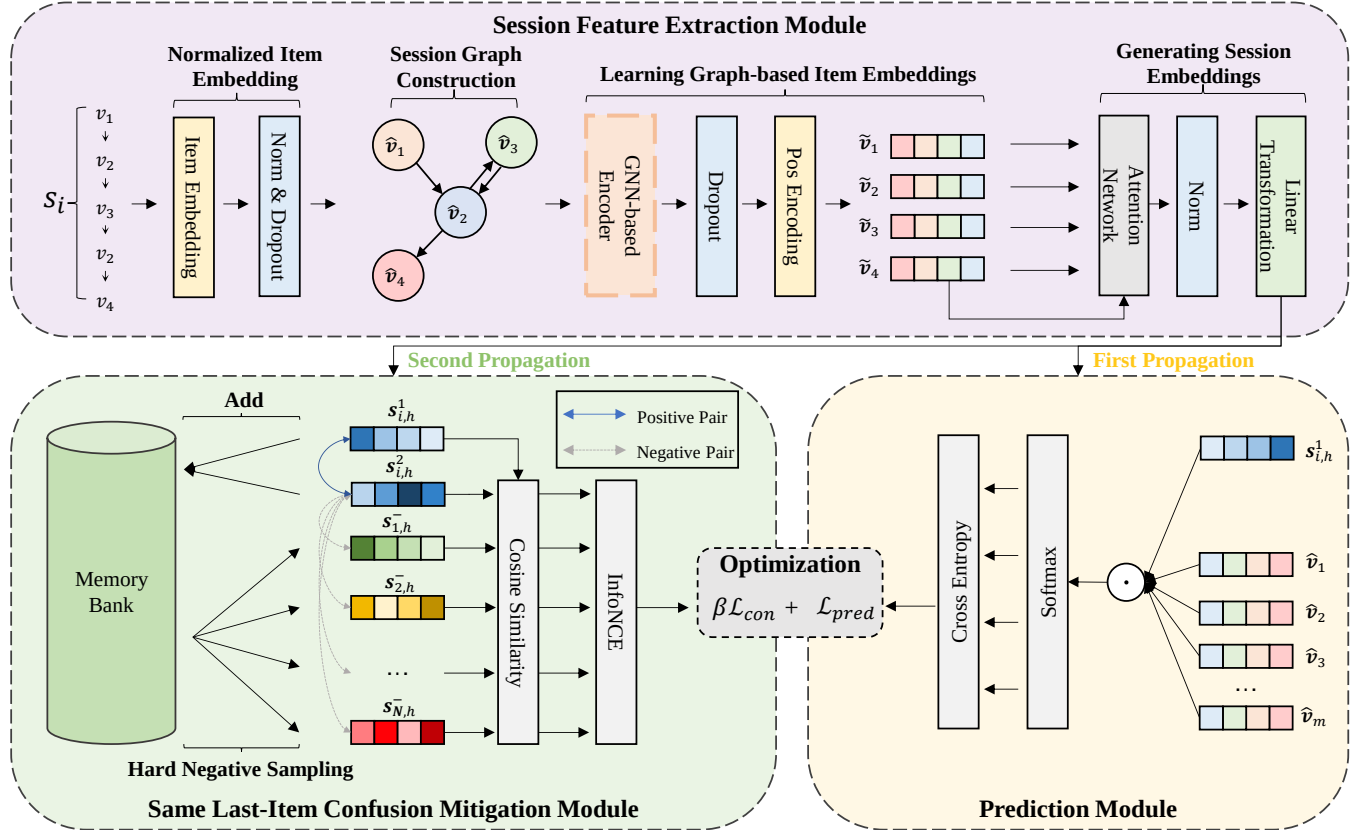


Figure 2: The overall network architecture of SAGE.

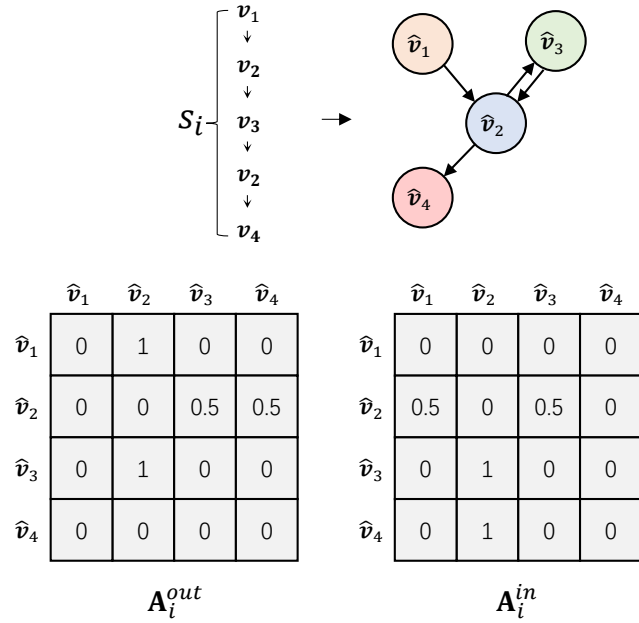


Figure 3: The demonstration of constructing a session graph, and its corresponding adjacency matrices.

use the previously obtained embedding vector $\hat{v}_i$ as the initial state of the nodes in the constructed session graph. After constructing the graph, we obtain the outgoing adjacency matrix $A_s^{out} \in \mathbb{R}^{d \times d}$ and incoming adjacency matrix $A_s^{in} \in \mathbb{R}^{d \times d}$, where d is the embedding dimension. By concatenating A_s^{out} and A_s^{in} , we obtain the final connection matrix $A_s \in \mathbb{R}^{d \times 2d}$ of the session graph.

3.3.3 Learning Graph-based Item Embeddings. Once the corresponding session graph is constructed, a graph neural network is required to extract structured information from it. In SAGE, we leverage a gated graph neural network (GGNN) to learn node vectors within a session graph. Notably, while different GNN-based SBR methods may vary in architectural details, most adopt the session graph construction approach introduced by SR-GNN. Formally, for node $v_{s,i}$ in graph $\mathcal{G}_s$, the update function can be formulated as,

$$\mathbf{a}_{s,i}^t = A_{s,i} \cdot \left[\mathbf{v}_1^{t-1}, \mathbf{v}_2^{t-1}, \dots, \mathbf{v}_{|S_i|}^{t-1} \right]^\top \mathbf{H} + \mathbf{b}, \quad (2)$$

$$\mathbf{z}_{s,i}^t = \sigma(\mathbf{W}_z \mathbf{a}_{s,i}^t + \mathbf{U}_z \mathbf{v}_i^{t-1}), \quad (3)$$

$$\mathbf{r}_{s,i}^t = \sigma(\mathbf{W}_r \mathbf{a}_{s,i}^t + \mathbf{U}_r \mathbf{v}_i^{t-1}), \quad (4)$$

$$\bar{\mathbf{v}}_i^t = \tanh(\mathbf{W}_o \mathbf{a}_{s,i}^t + \mathbf{U}_o (\mathbf{r}_{s,i}^t \odot \mathbf{v}_i^{t-1})), \quad (5)$$

$$\mathbf{v}_i^t = (1 - \mathbf{z}_{s,i}^t) \odot \mathbf{v}_i^{t-1} + \mathbf{z}_{s,i}^t \odot \bar{\mathbf{v}}_i^t, \quad (6)$$

where t is the training step, $A_{s,i}$ is the i -th row of matrix A_s corresponding to $v_{s,i}$, $\mathbf{H} \in \mathbb{R}^{d \times 2d}$ is trainable parameter, $\mathbf{z}_{s,i}^t$ and $\mathbf{r}_{s,i}^t$ are

the reset and update gates respectively, $[v_1^{t-1}, v_2^{t-1}, \dots, v_{|s_i|}^{t-1}]$ is the list of item vectors in s , $\sigma(\cdot)$ is the sigmoid function and $\odot$ is the Hadamard product operator.

To add the necessary noise to the resulting graph-based item representation vector, we also applied Dropout to the final output vector v_i^l after l -layers of GNN. Also, since there is no location information embedded in the graph neural network, we do the usual positional embedding on the generated item vectors. The final graph-based item embeddings $\tilde{v}_i$ can be calculated as,

$$\tilde{v}_i = \mathbf{w}_{pos(v_i)} + \text{Dropout}(v_i, p), \quad (7)$$

where $\mathbf{w}_t \in \mathbb{R}^d$ is the trainable positional embedding, $pos(v_i)$ is the absolute position of the item in the session, p is the Dropout probability.

3.3.4 Generating Session Embeddings. In this section, we consider constructing the session representation vector by combining long-term preference and short-term preference.

First, intuitively, we can represent the session's short-term preference by its last-interacted item $v_{s,|s_i|}$. Thus, for session $s_i = [v_{s,1}, v_{s,2}, \dots, v_{s,n}]$, the short-term session embedding $s_{i,s}$ of session s can be defined as $\tilde{v}_n$.

As for the long-term preference, we consider the long-term session embedding of session graph $\mathcal{G}_s$ by aggregating all node vectors. Meanwhile, as different interaction records may have different levels of priority, we utilize the attention mechanism to gain the long-term session preference $s_{i,l}$ as follows,

$$\alpha_i = \text{softmax}(\mathbf{q}^\top \sigma(\mathbf{W}_1 \tilde{v}_n + \mathbf{W}_2 \tilde{v}_i + \mathbf{b})), \quad (8)$$

$$s_{i,l} = \sum_{i=1}^n \alpha_i \tilde{v}_i, \quad (9)$$

where $\mathbf{q} \in \mathbb{R}^d$ and $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$ are trainable weights.

Finally, we get the final hybrid embedding $s_{i,h}$ by simply linear transform over the concatenation of the short-term and the long-term session embeddings.

$$s_{i,h} = \mathbf{W}_3 [s_{i,l}; s_{i,s}], \quad (10)$$

where $\mathbf{W}_3 \in \mathbb{R}^{2d \times d}$ is trainable parameter and $[\cdot; \cdot]$ represents concatenation operation.

3.4 Prediction Module

After extracting session hybrid embeddings, we adopt an MF layer to predict the relevance between the given session s_i and each candidate item $v_n \in V$ by multiplying the normalized session representation $Norm(s_{i,h})$ and normalized item embedding $Norm(v_n)$ for the avoidance of popularity bias, which can be defined as,

$$\hat{h}_n = r * Norm(s_{i,h}) * Norm(v_n), \quad (11)$$

As $Norm(s_{i,h}) * Norm(v_n)$ is equal to the cosine similarity between $s_{i,h}$ and v_n , the predicted logits are restricted to $[-1, 1]$, and the softmax score is likely to get saturated at high values for the training set. To this end, we add a scaling factor $r > 1$, which is useful in practice to allow for better convergence.

Then we apply a softmax function on the output logit to get the final scaled output probability vector $\hat{\mathbf{y}}$,

$$\hat{\mathbf{y}} = \text{softmax}(\hat{\mathbf{h}}), \quad (12)$$

where $\hat{\mathbf{h}}$ denotes the recommendation scores of all candidate items $v_n \in V$.

The prediction loss $\mathcal{L}_{pred}$ is defined by calculating the cross-entropy of the prediction and ground truth,

$$\mathcal{L}_{pred} = - \sum_{i=1}^{|D|} \mathbf{y}_i \log(\hat{\mathbf{y}}) + (1 - \mathbf{y}_i) \log(1 - \hat{\mathbf{y}}), \quad (13)$$

where $\mathbf{y}$ denotes the one-hot encoding vector for the ground truth item.

3.5 Same Last-Item Confusion Mitigation Module

As we mentioned before, simply using the hybrid session embedding $s_{i,h}$ for item prediction inevitably leads to the same last-item confusion problem.

To address this, the most intuitive idea is to separate session representations with the same last item as much as possible. Naturally, we came up with the idea of using comparative learning to solve this problem. We refer to the schema of [10], and since we have added the Dropout layer above, we can simply perform twice forward propagation for each session s_i to produce two hybrid embeddings. For the embedding vector obtained by the first forward propagation, we denote it by $s_{i,h}^1$ and the second by $s_{i,h}^2$, which constructs a "positive pair". For negative samples, we implement a hard negative sampling strategy based on the Jaccard similarity between sessions. Specifically, we select sessions that share the same last item as the current session but exhibit low Jaccard similarity with the current session's items. If the number of such hard negatives is insufficient, we supplement them with randomly selected sessions. These negative samples are represented by $s_{j,h}^-$, $j \in 1, 2, \dots, N$, where N indicates the number of negative samples. In this way, the contrastive loss $\mathcal{L}_{con}$ is calculated as,

$$\mathcal{L}_{con} = - \sum_{i=1}^{|D|} \log \frac{e^{\text{sim}(s_{i,h}^1, s_{i,h}^2)/\tau}}{\sum_{j=1}^N (e^{\text{sim}(s_{i,h}^1, s_{j,h}^-)/\tau} + e^{\text{sim}(s_{i,h}^2, s_{j,h}^-)/\tau})}, \quad (14)$$

where τ is the temperature hyperparameter, D is the training dataset, and we utilize cosine similarity for $\text{sim}(\cdot, \cdot)$ as follows,

$$\text{sim}(s_1, s_2) = \frac{s_1 s_2}{\|s_1\| \|s_2\|} \quad (15)$$

However, unlike [10], since there are not always adequate negative sessions with the same last item in the same training batch, we cannot directly use sessions in the same training batch as negative samples. If we forward the corresponding negative sessions each time we calculate the contrastive loss, it would result in a huge waste of computational resources. As a result, instead of exhaustively computing these representations, similar to [38], we maintain a memory bank $M = \{(f_i^1, f_i^2)\}$. During each learning iteration, the two hybrid embeddings $s_{i,h}^1, s_{i,h}^2$ are updated to M at the corresponding session entry $s_{i,h}^1 \rightarrow f_i^1, s_{i,h}^2 \rightarrow f_i^2$, and negative session embeddings can be sampled from M , i.e., $s_{j,1,h}^- \leftarrow f_j^1, s_{j,2,h}^- \leftarrow f_j^2$.

This allows us to compute the comparison loss without additional forward propagation, but only with the corresponding vector in the memory bank M .

3.6 Model Optimization

In the previous section, we defined the prediction loss $\mathcal{L}_{pred}$ and the contrastive loss $\mathcal{L}_{con}$, we define our final loss function with the integration of both of them as follows:

$$\mathcal{L} = \mathcal{L}_{pred} + \beta \mathcal{L}_{con} + \lambda \|\Theta\|_2^2, \quad (16)$$

where Θ is all trainable parameters, β is the hyper-parameter for balancing the contrastive module, λ is the regularization hyper-parameter. The whole training procedure of the proposed *SAGE* framework is summarized in Algorithm 1.

Algorithm 1: Training procedure of *SAGE*

Input: Training sessions $\mathcal{S}$, item set $\mathcal{V}$
Output: Recommendation list per session

- 1 Initialize item embeddings v_i with L_2 normalization and dropout (Eq. 1);
- 2 Initialize memory bank $M = \{(f_i^1, f_i^2)\}$;
- 3 **for** *epoch* **do**
- 4 **for** *batch* $\mathcal{B} \subset \mathcal{S}$ **do**
- 5 **for** *session* $s_i \in \mathcal{B}$ **do**
- 6 Construct session graph $\mathcal{G}_s = (\mathcal{V}_s, \mathcal{E}_s)$;
- 7 Learn item embeddings via GGNN (Eq. 2–6);
- 8 Apply dropout and positional embedding to item embeddings (Eq. 7);
- 9 Compute hybrid session embedding $s_{i,h}$ (Eq. 8–10);
- 10 Generate two views: $s_{i,h}^1, s_{i,h}^2$ via dropout;
- 11 **end**
- 12 Compute prediction loss $\mathcal{L}_{pred}$ (Eq. 11–13);
- 13 Update memory bank M with $s_{i,h}^1, s_{i,h}^2$;
- 14 Sample hard negatives $s_{j,h}^-$ from memory bank M ;
- 15 Compute contrastive loss $\mathcal{L}_{con}$ (Eq. 14);
- 16 Jointly optimize the final objectives $\mathcal{L}$ (Eq. 16);
- 17 **end**
- 18 **end**

4 Experiments

4.1 Datasets

Following previous work, we choose two commonly used session recommendation datasets, that is, Yoochoose 1/64 and Diginetica. Yoochoose dataset is from the RecSys Challenge 2015 and consists of six months of interaction sessions from an E-commercial website. We only make use of the most recent fractions 1/64 of the training sequences of Yoochoose denoted. Diginetica dataset comes from CIKM Cup 2016, and only the transaction data is used.

For fairness consideration, we use the same preprocessing techniques as [37], which filter out all sessions of length 1 and items that appear less than 5 times in all datasets. Moreover, we do data

augmentation on a session to obtain sequences and corresponding labels. For example, for session $s = [v_1, v_2, \dots, v_n]$, we split it in to $([v_1], v_2), ([v_1, v_2], v_3), \dots, ([v_1, v_2, \dots, v_{n-1}], v_n)$. The statistics of datasets are shown in Table 1.

Table 1: Statistics of the datasets.

Dataset	Yoochoose 1/64	Diginetica
# Click	557,248	982,961
# Training sessions	369,859	719,470
# Test sessions	55,898	60,858
# Items	16,766	43,097
Average Length	6.16	5.12

4.2 Evaluation Metrics

We use the same metrics, Recall@K and Mean Reciprocal Rank (MRR@K), as in [37] with K = 20.

4.3 Baselines

To evident the effectiveness of our proposed *SAGE*, we compare it with the following representative baselines.

- *POP* and *SPOP* recommend the top-K popular items in the training dataset and the current predicting session, respectively.
- *Item-KNN* [28] leverages item-to-item collaborative filtering techniques, which recommend items similar to the previously interacted items by cosine similarity.
- *BPR* [26] is a classical matrix factorization (MF) method, and is optimized by a pairwise ranking loss function.
- *FPMC* [27] is a sequential prediction method combining the Markov chain and MF.
- *GRU4REC* [18] utilizes the RNN structure to model the sequential interaction of users and leverages multiple tricks to help the RNN converge to the session recommendation problem.
- *NARM* [19] improves GRU4REC by incorporating an attention mechanism into RNN.
- *STAMP* [21] replaces the RNN structures by employing attention mechanism.
- *SR-GNN* [37] first introduces session graph structure into a session-based recommendation. By utilizing a Gated GNN to extract on-graph item embeddings, the last item together with a weighted sum of all the session embeddings are concatenated for prediction.
- *FGNN* [25] formulates the next item recommendation within the session as a graph classification problem.
- *TAGNN* [41] introduces target-aware attention to jointly consider item transitions and user interests for specific target items.
- *GCE-GNN* [35] constructed a global graph using session data and modified the model structure to introduce the global information learned from the global graph.
- *Disen-GNN* [17] proposes a disentangled graph neural network to capture the session purpose.

- S^2 -DHCN [39] constructs each session as a hypergraph and utilizes contractive learning methods.
- DMI-GNN [23] introduces multi-positional encoding and dynamic regularization to mitigate excessive interests in SBR.

4.4 Implementation Details

To align with the previous work, we set the hidden size $d = 100$, while model parameters are initialized using a Gaussian distribution with a mean of 0 and a deviation of 0.1. The mini-batch Adam optimizer is utilized to optimize model parameters. The initial learning rate is set to $1e-3$ and decays by 0.1 every 3 epochs. Dropout probabilities for all dropout layers are set to 0.1. For the contrastive module, the temperature parameter τ is set to 12, and β is set to 0.1 for Yoochoose 1/64 and 1 for Diginetica. All the hyperparameters are tuned on the validation set. Experiments ran on an Ubuntu 16.04.7 server with 24-core Intel® Xeon® E5-2680 v3 CPUs, 125GB RAM, and five NVIDIA GPUs (four RTX 2080 Ti + one RTX 4090). We used Python 3.8.19 with PyTorch 1.12.1, CUDA 11.6, and numpy 1.24.4.

4.5 Experiment Results

Table 2: Recommendation performance on public datasets (%). The best-performing method in each column is boldfaced, and the second-best method is underlined.

Method	Yoochoose 1/64		Diginetica	
	Recall@20	MRR@20	Recall@20	MRR@20
POP	6.71	1.65	0.89	0.20
S-POP	30.44	18.35	21.06	13.68
Item-KNN	51.60	21.81	35.75	11.57
BPR	31.31	12.08	5.24	1.98
FPMC	45.62	15.01	26.53	6.95
GRU4REC	60.64	22.89	29.45	8.33
NARM	68.32	28.63	49.70	16.17
STAMP	68.74	29.67	45.64	14.32
SR-GNN	70.57	30.94	50.73	17.59
FGNN	71.75	<u>31.71</u>	51.36	18.47
TAGNN	71.02	31.12	51.31	18.03
GCE-GNN	70.91	30.63	54.22	<u>19.04</u>
Disen-GNN	71.46	31.36	53.79	18.99
S^2 -DHCN	70.39	29.92	53.66	18.51
DMI-GNN	<u>72.25</u>	30.74	<u>54.25</u>	19.21
<i>SAGE</i> (SR-GNN)	71.61	31.99	54.01	<u>19.04</u>
<i>SAGE</i> (TAGNN)	71.72	30.78	53.01	18.44
<i>SAGE</i> (GCE-GNN)	72.29	30.88	54.26	<u>19.04</u>

To further demonstrate the overall performance of our proposed *SAGE*, we compare it with the selected baselines described above. The experimental results are shown in Table 2. From Table 2, we can see that our *SAGE* achieves the best performance on both datasets, especially in MRR@20, which illustrates the superior ranking capability compared with other baseline methods.

Among traditional methods, the Item-KNN achieves the best performance, although the overall performance of all traditional

methods is relatively poor. To our surprise, the simple yet effective S-POP shows better performance than those of BPR and FPMC. Notably, the S-POP considers only item popularity, which means both the Yoochoose 1/64 dataset and the Diginetica dataset suffer from popularity bias. It’s also vital to point out that the Item-KNN utilizes pairwise item similarities only and performs better than FPMC, which is an MC-based approach with the assumption that only the last interacted items are needed to perform sequential recommendation. This phenomenon indicates that the simple MC assumption is not suitable for such complex sessions.

For deep learning-based methods, all DL-based methods consistently outperform traditional methods. GRU4REC and NARM, both based on RNN structures, achieve decent performances. NARM, which adds an attention mechanism to the original RNN, significantly outperforms GRU4REC. STAMP, which replaces RNN with attentional MLPs, shows comparable performance to NARM. Finally, all DL-based methods share better performance over FPMC, which shows the importance of modeling the whole interaction sequence instead of considering merely the last click. However, neither RNNs nor MLPs are suitable for capturing complex transitions among sessions. This may explain why they are worse than graph-based methods.

Graph-based methods significantly outperform other baselines. SR-GNN, which introduces session graph structures, achieves better performance than most other methods. FGNN and TAGNN further enhance performance by rethinking item order and considering target item features, respectively. GCE-GNN leverages global information to improve upon SR-GNN, achieving high performance on both datasets. Disen-GNN and S^2 -DHCN also show strong performance, highlighting the effectiveness of graph-based approaches.

Our *SAGE* framework, when integrated with these graph-based methods, consistently achieves the best performance. Specifically, *SAGE*(SR-GNN) outperforms SR-GNN in both Recall@20 and MRR@20 on both datasets, demonstrating the effectiveness of our framework in enhancing the base model. *SAGE*(TAGNN) also shows significant improvements over TAGNN, achieving better performance in both Recall@20 and MRR@20 on Diginetica. Moreover, *SAGE*(GCE-GNN) achieves the best performance among all methods on both datasets across both metrics, except MRR@20 on the Yoochoose 1/64 dataset. This further confirms the generalizability and effectiveness of *SAGE* in enhancing GNN-based SBR methods by mitigating the same last-item confusion problem through contrastive learning.

4.6 Ablation Study

To validate module contributions in *SAGE*, we compare full implementations on SR-GNN and TAGNN backbones against four variants:

- (1) **SAGE-Contrast.** We removed the contrastive module of *SAGE*.
- (2) **SAGE-HardNeg.** We randomly sampled negative sessions instead of choosing sessions with the same last item.
- (3) **SAGE-Norm.** We removed all normalizations.
- (4) **SAGE-PE.** We removed positional embeddings.

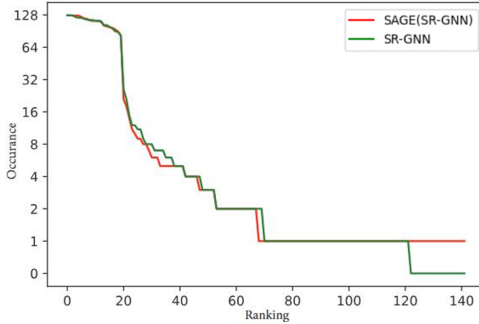
Results (Table 3) demonstrate critical dependencies: Removing contrastive learning consistently degrades performance across

Table 3: Performance of variants of SAGE(SR-GNN) and SAGE(TAGNN) (%).

Method	Yoochoose 1/64		Diginetica	
	Recall@20	MRR@20	Recall@20	MRR@20
SR-GNN	70.57	30.94	50.73	17.59
SAGE(SR-GNN)	<u>71.61</u>	31.99	54.01	19.04
-Contrast	71.31	<u>31.80</u>	53.49	<u>19.01</u>
-HardNeg	71.65	31.16	<u>53.99</u>	18.92
-Norm	71.01	31.55	51.77	17.58
-PE	71.39	30.92	53.80	18.94
TAGNN	71.02	31.12	51.31	18.03
SAGE(TAGNN)	71.72	<u>30.78</u>	53.01	18.44
-Contrast	71.54	30.69	52.53	<u>18.43</u>
-HardNeg	<u>71.61</u>	30.51	52.77	18.31
-Norm	71.08	30.64	52.22	18.21
-PE	71.57	30.42	<u>52.89</u>	18.42

datasets and base models, confirming its essential role in session representation enhancement; Hard negative sampling proves vital for ranking metrics, particularly in TAGNN, where performance universally declines; Normalization removal severely damages overall performance by impairing popularity bias suppression and contrastive learning; Positional embeddings significantly impact ranking outcomes, with MRR@20 on Yoochoose 1/64 showing marked deterioration upon their removal. Collectively, these findings validate the necessity of all proposed components in SAGE.

4.7 Case Study

**Figure 4: Predicted results of SAGE(SR-GNN) and SR-GNN on sessions with the same last-item (item 33889).**

4.7.1 Effect on Solving Same Last-Item Confusion. Consistent with the Introduction section, we still take sessions with the same last item (item 33889) and utilize the trained SR-GNN and SAGE(SR-GNN) to provide predictions. For each session, 20 items are recommended and counted. The relationship between the number of occurrences of an item in the prediction candidate set and its corresponding occurrence ranking is shown in Fig. 4. From the figure, we can see that SAGE(SR-GNN) predicts a lower curve of item occurrences compared to SR-GNN, which shows that SAGE(SR-GNN) can provide different recommendations for sessions with the same last item. At the same time, SAGE(SR-GNN) recommended a total of

142 kinds of items, 20 more than the 122 kinds of items in SR-GNN, which also confirms the effectiveness of our method in solving the same last-item confusion.

4.7.2 Effect on Solving Popularity Bias. To measure the popularity of recommended items, we calculated **Average Recommendation Popularity (ARP)** on SR-GNN and SAGE(SR-GNN) to compare the difference in popularity of recommended items between the two methods. The ARP can be calculated as follows,

$$ARP = \frac{1}{|S|} \sum_{s \in S} \frac{\sum_{i \in L_s} \phi(i)}{K}, \quad (17)$$

where $\phi(i)$ is the number of times that item i appears in the training dataset, L_s is the recommended item list for session s , and S is the set of sessions in the test dataset.

Table 4: SAGE(SR-GNN) versus SR-GNN in terms of Average Recommendation Popularity (ARP). A lower ARP value indicates a lower popularity bias.

Method	Yoochoose 1/64	Diginetica
SR-GNN	4128.54	495.25
SAGE(SR-GNN)	2678.64	285.85

From Table 4, it is easy to see that SAGE(SR-GNN) has a huge difference in the popularity of recommended items on both datasets compared to SR-GNN. It proves that our SAGE framework can indeed somewhat solve the popularity bias problem.

5 Conclusion

In this paper, we proposed the unified mitigation framework SAGE to address two key limitations of GNN-based session-based recommendation (SBR) models. Specifically, SAGE uses a contrastive module to resolve "same last-item confusion" (indistinguishable representations for sessions with the same last item) and adopts normalized item/session embeddings to alleviate popularity bias (over-recommending popular items). Experiments on two real-world datasets demonstrated SAGE effectively boosts GNN-based SBR performance and is compatible with mainstream architectures. Future work will explore techniques to further reduce the last item's influence on session representations and enhance SBR system's robustness.

6 Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (62572075, 62572217, 62306287), in part by the Beijing Natural Science Foundation (L233034, L253004), in part by Fundamental Research Funds for the Beijing University of Posts and Telecommunications (No.2025TSQY01), in part by Science and Technology Program of Guangzhou (2024A04J6317), in part by State Key Laboratory of Radio Frequency Heterogeneous Integration (No.2025006), in part by Scientific Foundation for Youth Scholars of Shenzhen University (No.868-000001032902) and part by Natural Science Foundation of Guangdong Province (2025A1515010739).

References

- [1] Shiyu Chang, Wei Han, Jiliang Tang, Guo-Jun Qi, Charu C Aggarwal, and Thomas S Huang. 2015. Heterogeneous network embedding via deep architectures. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 119–128.
- [2] Jinpeng Chen, Yuan Cao, Fan Zhang, Pengfei Sun, and Kaimin Wei. 2022. Sequential intention-aware recommender based on user interaction graph. In *Proceedings of the 2022 International Conference on Multimedia Retrieval*. 118–126.
- [3] Ming Chen, Weike Pan, and Zhong Ming. 2024. Explicit and Implicit Modeling via Dual-Path Transformer for Behavior Set-informed Sequential Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 329–340.
- [4] Tianwen Chen and Raymond Chi-Wing Wong. 2020. Handling information loss of graph neural networks for session-based recommendation. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1172–1180.
- [5] Minjin Choi, Hye-young Kim, Hyunsouk Cho, and Jongwuk Lee. 2024. Multi-intent-aware Session-based Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2532–2536.
- [6] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. 2021. Transformers4rec: Bridging the gap between nlp and sequential/session-based recommendation. In *Proceedings of the 15th ACM conference on recommender systems*. 143–153.
- [7] Yuxiao Dong, Ziniu Hu, Kuansan Wang, Yizhou Sun, and Jie Tang. 2020. Heterogeneous Network Representation Learning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, Christian Bessiere (Ed.), 4861–4867.
- [8] Yingpeng Du, Ziyang Wang, Zhu Sun, Yining Ma, Hongzhi Liu, and Jie Zhang. 2024. Disentangled Multi-interest Representation Learning for Sequential Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 677–688.
- [9] Kairui Fu, Shengyu Zhang, Zheqi Lv, Jingyuan Chen, and Jiwei Li. 2024. DIET: Customized slimming for incompatible networks in sequential recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 816–826.
- [10] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. Simcse: Simple contrastive learning of sentence embeddings. *arXiv preprint arXiv:2104.08821* (2021).
- [11] Linxin Guo, Yaochen Zhu, Min Gao, Yinghui Tao, Junliang Yu, and Chen Chen. 2024. Consistency and Discrepancy-Based Contrastive Tripartite Graph Learning for Recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 944–955.
- [12] Priyanka Gupta, Diksha Garg, Pankaj Malhotra, Lovkesh Vig, and Gautam M Shroff. 2019. NISER: normalized item and session representations with graph neural networks. *arXiv preprint arXiv:1909.04276* 43 (2019).
- [13] Ruining He and Julian McAuley. 2016. Fusing similarity models with markov chains for sparse sequential recommendation. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 191–200.
- [14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [15] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [16] Sara Latifi, Noemi Mauro, and Dietmar Jannach. 2021. Session-aware recommendation: A surprising quest for the state-of-the-art. *Information Sciences* 573 (2021), 291–315.
- [17] Ansong Li, Zhiyong Cheng, Fan Liu, Zan Gao, Weili Guan, and Yuxin Peng. 2023. Disentangled Graph Neural Networks for Session-Based Recommendation. *IEEE Trans. Knowl. Data Eng.* 35, 8 (2023), 7870–7882. doi:10.1109/TKDE.2023.3208782
- [18] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1419–1428.
- [19] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1419–1428.
- [20] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493* (2015).
- [21] Qiao Liu, Yifu Zeng, Refuoe Mokhosi, and Haibin Zhang. 2018. STAMP: short-term attention/memory priority model for session-based recommendation. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1831–1839.
- [22] Yuli Liu, Christian Walder, Lexing Xie, and Yiqun Liu. 2024. Probabilistic Attention for Sequential Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1956–1967.
- [23] Mingyang Lv, Xiangfeng Liu, and Yuanbo Xu. 2025. Dynamic Multi-Interest Graph Neural Network for Session-Based Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12328–12336.
- [24] Andriy Mnih and Russ R Salakhutdinov. 2007. Probabilistic matrix factorization. *Advances in neural information processing systems* (2007), 1257–1264.
- [25] Ruihong Qiu, Jingjing Li, Zi Huang, and Hongzhi Yin. 2019. Rethinking the item order in session-based recommendation with graph neural networks. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 579–588.
- [26] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian personalization ranking from implicit feedback. *arXiv preprint arXiv:1205.2618* (2012).
- [27] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized markov chains for next-basket recommendation. In *Proceedings of the 19th international conference on World wide web*. 811–820.
- [28] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*. 285–295.
- [29] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research* 15, 1 (2014), 1929–1958.
- [30] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [31] Zhu Sun, Hongyang Liu, Xinghua Qu, Kaidong Feng, Yan Wang, and Yew Soon Ong. 2024. Large language models for intent-driven session recommendations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 324–334.
- [32] Yong Kiam Tan, Xinxing Xu, and Yong Liu. 2016. Improved recurrent neural networks for session-based recommendations. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 17–22.
- [33] Jiakai Tang, Sunhao Dai, Zexu Sun, Xu Chen, Jun Xu, Wenhui Yu, Lantao Hu, Peng Jiang, and Han Li. 2024. Towards Robust Recommendation via Decision Boundary-aware Graph Contrastive Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2854–2865.
- [34] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.
- [35] Ziyang Wang, Wei Wei, Gao Cong, Xiao Li-Li, Xian-Ling Mao, and Minghui Qiu. 2020. Global context enhanced graph neural networks for session-based recommendation. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 169–178.
- [36] Zongwei Wang, Junliang Yu, Min Gao, Hongzhi Yin, Bin Cui, and Shazia Sadiq. 2024. Unveiling Vulnerabilities of Contrastive Recommender Systems to Poisoning Attacks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3311–3322.
- [37] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-based recommendation with graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 346–353.
- [38] Zhirong Wu, Yuanjun Xiong, Stella X Yu, and Dahua Lin. 2018. Unsupervised feature learning via non-parametric instance discrimination. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3733–3742.
- [39] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zhang. 2021. Self-supervised hypergraph convolutional networks for session-based recommendation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4503–4511.
- [40] Chengfeng Xu, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Fuzhen Zhuang, Junhua Fang, and Xiaofang Zhou. 2019. Graph contextualized self-attention network for session-based recommendation. In *IJCAI*, Vol. 19. 3940–3946.
- [41] Feng Yu, Yanqiao Zhu, Qiang Liu, Shu Wu, Liang Wang, and Tieniu Tan. 2020. TAGNN: Target attentive graph neural networks for session-based recommendation. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 1921–1924.
- [42] Xiaokun Zhang, Bo Xu, Zhaochun Ren, Xiaochen Wang, Hongfei Lin, and Fenglong Ma. 2024. Disentangling id and modality effects for session-based recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1883–1892.

Appendix

In this appendix, we introduce a table of key notations used in this paper due to the space limitation.

A Notation Table

The notations used in this paper are summarized in Table 5.

Table 5: Main notations used in this work.

Notation	Description
$\mathcal{V} = \{v_1, v_2, \dots, v_m\}$	The set of all items.
$\mathcal{S}$	The set of all sessions.
$s_i = [v_1^i, v_2^i, \dots, v_{ s_i }^i]$	The i -th session, represented as a sequence of items.
v_j^i	The j -th item in the i -th session.
$\mathbf{v}_i \in \mathbb{R}^d$	The d -dimensional embedding of item v_i .
$\hat{\mathbf{v}}_i$	The normalized item embedding with dropout applied.
$\mathcal{G}_s = (\mathcal{V}_s, \mathcal{E}_s)$	The session graph constructed for session s , where $\mathcal{V}_s$ is the set of items and $\mathcal{E}_s$ is the set of edges representing user interactions.
$\mathbf{A}_s \in \mathbb{R}^{d \times 2d}$	The concatenated adjacency matrix of the session graph.
$\mathbf{A}_s^{out}, \mathbf{A}_s^{in}$	The outgoing and incoming adjacency matrices of the session graph.
$\mathbf{v}_i^t$	The representation of node v_i at training step t in the GGNN.
$\mathbf{a}_{s,i}^t, \mathbf{z}_{s,i}^t, \mathbf{r}_{s,i}^t$	The aggregated neighbor information, update gate, and reset gate for node v_i at step t in GGNN.
$\tilde{\mathbf{v}}_i$	The final graph-based item embedding with positional and dropout augmentation.
$\mathbf{s}_{i,s}, \mathbf{s}_{i,l}, \mathbf{s}_{i,h}$	Short-term, long-term, and hybrid session embeddings, respectively.
α_i	The attention weight for aggregating node vectors to form the long-term session embedding.
$\hat{h}_n$	The predicted relevance score between session s_i and candidate item v_n .
$\hat{\mathbf{y}}$	The final scaled output probability vector for item predictions.
$\mathcal{L}_{pred}$	The prediction loss is calculated using cross-entropy between predicted scores and ground truth.
$\mathcal{L}_{con}$	The contrastive loss for mitigating same last-item confusion.
τ	The temperature hyperparameter for contrastive loss calculation.
$M = \{(f_i^1, f_i^2)\}$	The memory bank storing session embeddings for negative sampling.
$\mathcal{L}$	The final loss function combining prediction loss, contrastive loss, and L2 regularization.
β, λ	Hyperparameters for balancing contrastive loss and regularization, respectively.